

ARI: 自律型研究基盤

樹神 宇徳

v0.8.1, 2026-06-01

Abstract

本レポートでは、木探索ベースの探索ループと論文生成パイプラインを組み合わせた自律型研究基盤 **ARI** を述べる。探索ループは研究ステップノードに対する最良優先木探索 (BFTS) であり、ノード拡張は Reason-and-Act (ReAct [1]) 形式のエージェントが 14 のドメインスキルレジストリにツール呼び出しを発する形で行う。論文生成パイプラインは生き残った系譜から草稿を生成し、階層型ルブリックの下で反復的に改稿する。すべての成果物は単一のチェックポイントディレクトリに格納され、これが再現性、系譜、コスト計上の単位となる。本レポートはアルゴリズムと設計上の論拠を形式化し、予備的観察を報告し、設計を制約する決定性と新規性のトレードオフを議論する。

1 はじめに

1.1 動機

機械学習および周辺領域の研究ワークフローには共通の骨格がある: 研究アイデアを発想し、実験を設計・実行し、結果を分析し、論文を執筆し、そして反復する。各ステップは大規模言語モデルにとってますます扱いやすくなっているが、それらを単一の自律ループへ縫合することは、これまで未解決のエンジニアリング課題であった。最も近接する既発表システム — AIDE、AI Scientist ファミリ、VirSci、PaperBench 評価器 — は、それぞれ 1~2 軸を埋めはするが、残りの軸は人手に任せている。

ARI はこの空隙を埋めるために存在する。本稿ではユーザが研究問題と計算予算を与えると仮定する; システムは、発表可能な論文、各主張を裏付ける実験コード、そして到達経路の完全な監査記録を返す。監査記録は譲歩できない要件である: 探索木の各ノード、各ツール呼び出し、各モデル呼び出し、そして API 利用の各ドルが、単一のチェックポイントディレクトリ内に記録される ([2] は同様の規律を採用している)。チェックポイントは再現性の単位である。

1.2 貢献

本レポートは以下の三つの貢献を形式化する:

- 研究ステップノード上の**最良優先木探索**であり、その選択スコア $U(n)$ は経験報酬、新規性項、深さペナルティを分離している (section 3.1)。
- 階層型ルブリック R をキーに据える**論文生成パイプライン**; 葉判官が個々の主張を採点し、システムスコアは $\text{score}(P) = \sum_i w_i \text{judge}_i(P)$ となる (section 4.3)。PaperBench 複製器はツール呼び出しとして統合される (section 4.5)。
- パイプラインを完全に再現可能にする**チェックポイントスコープ**の規律: すべての外部状態 (メモリ、コスト台帳、系譜ポイント) は `$ARI_CHECKPOINT_DIR` 配下書かれ、ユーザの

ホームディレクトリには決して書き出されない。これは**決定性**という設計原則の運用上の帰結である (section 2.5 で 5 つの原則を完全に提示する; 本報告ではこの原則を P2 と呼ぶ)。

1.3 本レポートの適用範囲

本レポートは `ari-core` および 14 個の `ari-skill-*` パッケージの v0.8.0 を記述する。v0.8.0 では、PaperBench の再現経路をプロトコル忠実な三段契約 — エージェントロールアウト、再現、採点 — へ格上げし、ロールアウトエージェントがホストの提供しない能力を仮定しないようにするホスト真実の環境ガードレールを加え、探索評価層を設定可能にし、実対象である非可逆圧縮論文に対する初のオペレーショナル再現パスを実施した (section 5)。本文はアルゴリズムとデータフローをモジュール水準で記述し、システムが用いる LLM プロンプトの逐語掲載は section B に集約する。本版における経験的研究は予備的である — section 5 はこれまでに得られた観察を報告し、制御されたベンチマークは今後の課題として残す。

1.4 構成

Section 2 は ARI のトップダウンビューを与える: オークストレータ、14 個のスキル、パイプライン DAG、設計原則。Sections 3 and 4 は二つの技術的核であり、それぞれ形式化された定義と経験的観察を含む。Section 5 は予備的観察を報告し; section 6 は本研究の位置づけ; section 7 は未解決問題を集約する。

2 システム概観

2.1 フルスタック

Figure 1 は v0.8.0 の ARI フルスタックを描く。システムは一つのドライバと多数のツール提供者に分かれる。ドライバ (`ari-core`) はオークストレータ、BFTS エンジン、系譜歩行器、エージェントループ、チェックポイント、コストトラッカ、設定ロードを保有する。各スキルは独立した Python パッケージとして *Model*

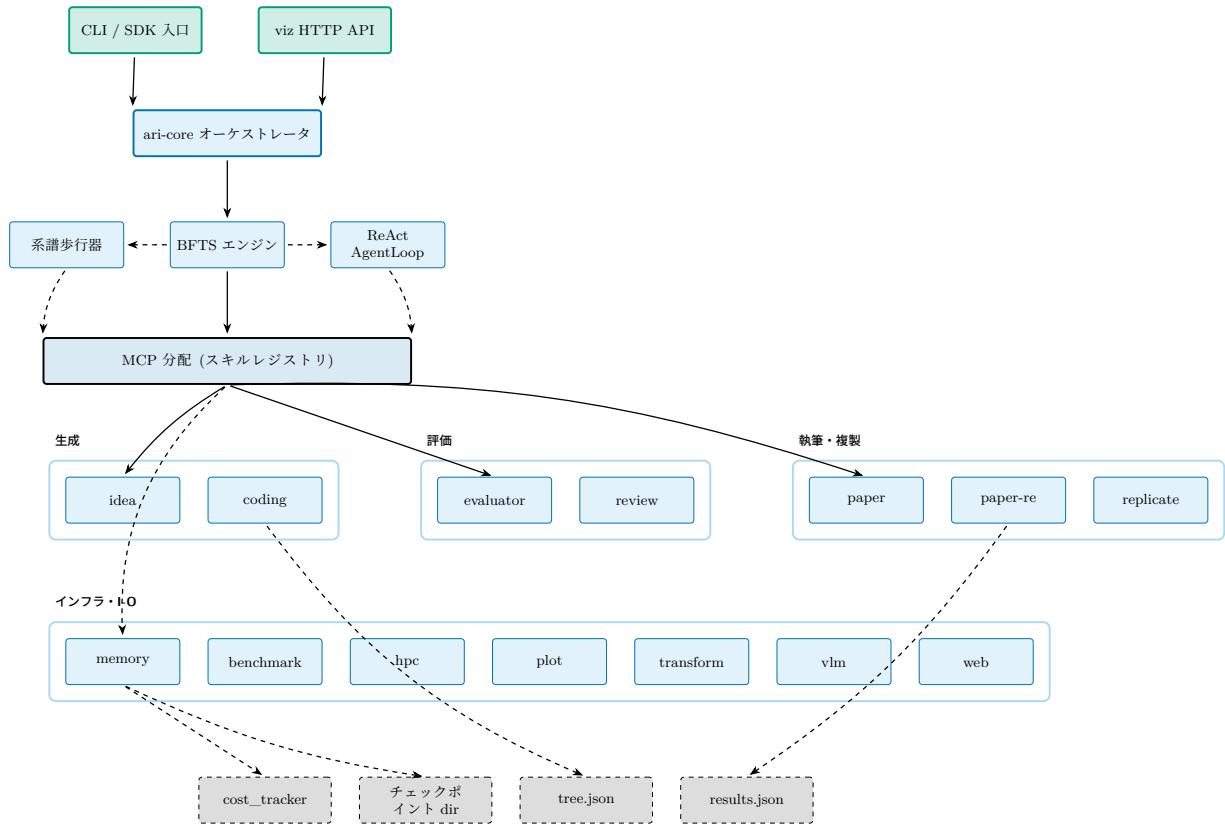


Figure 1: v0.8.0 における ARI のフルスタック。CLI / SDK 入口は単一のオーケストレータ (**ari-core**) を駆動する; オーケストレータは三つのエンジン (BFTS、系譜歩行者、ReAct AgentLoop) を所有し、MCP 層を通じて 14 スキルレジストリへ分配する。永続化はすべて単一のチェックポイントディレクトリに落ちる: コスト台帳、**tree.json**、**results.json**。実線は制御、破線は状態を表す。

Context Protocol(MCP) サーバ形式で出荷され、固有の **skill.yaml** 宣言を持つ。MCP は Anthropic が標準化した JSON-RPC-over-stdio インタフェースであり、LLM エージェントループに工具的機能を露出させるためのもの; ARI はこれを **ari-core** と各スキルの間の汎用継ぎ目として用いる。現在ツリーに含まれる 14 のスキル: **benchmark**、**coding**、**evaluator**、**hpc**、**idea**、**memory**、**orchestrator**、**paper**、**paper-re**、**plot**、**replicate**、**transform**、**vlm**、**web**。

オーケストレータは呼び出しの分配のみを担い、ドメイン作業自体は実行しない。この分離が重要なのは、スキルを追加・差し替えても探索アルゴリズムには触れずに済むからである; BFTS エンジンはどのスキルが参加するかを知らない; すべては MCP 分配層を介する。

Figure 2 は本レポートの残りで使う「運用ビュー」(制御 + 状態出力) であり、fig. 1 は「層別ビュー」(ドライバ → MCP → スキル → 永続化) である。

2.2 パイプライン DAG

一回の実行は四つのステーションのパイプラインに帰着する (fig. 3)。idea ステーションは研究問題を選択または生成し、exploration ステーションは BFTS の下でノードを派生させ、paper ステーションは最良系譜を草稿に編成し、review ステーションはルブリックで草稿を採点する。グラフは DAG にレビューから紙へのバックエッジを 1 本加えた構造である: このエッジのみが非単調性を生じうる (改稿が軸を悪化させ得る);

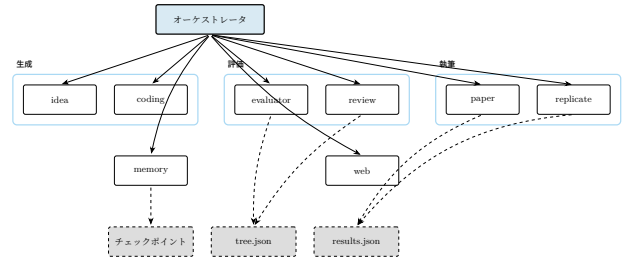


Figure 2: 運用ビュー: オーケストレータがスキルレジストリへ分配する; 各スキルはアクティブなチェックポイントディレクトリのみへ書き込む。破線は状態を表す。

収束基準 (section 4.4) がバックエッジ回数の上界を与える。

パイプラインはオーケストレータ内の単一の実行器が駆動する; 科学データ組立器が探索出力と論文入力の間隙を埋め、執筆器が必要とするノード単位の成果物を集約する。

2.3 BFTS 制御フロー

Figure 4 は 1 BFTS ステップを端から端まで追う。実装は section 3 で展開する; 図は本レポートの残りで使う語彙 (フロンティア、選択、フォールバック、expand、prune、停滞) と、各 LLM 主導ステップを実体化するプロンプトファイル (**bfts_select.md**、**bfts_expand_select.md**、**bfts_expand.md**) を確定する。

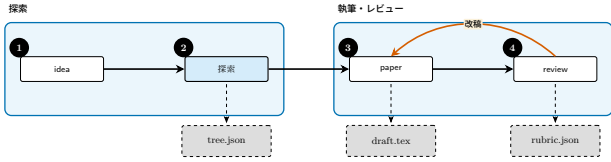


Figure 3: パイプライン DAG: idea → exploration → paper → review、ただし 改稿 がバックエッジを閉じる。破線出力はチェックポイントへ永続化される; 改稿サイクルは草稿を書き直し、レビューを再実行する。

2.4 paper-re コンポーネントビュー

Figure 5 は paper-re の二段階目のビューである。この層での ARI の貢献は小さいが具体的である: 薄いアダプタが各エージェントツールを包んで出力をサイズ制限し、vendoring されたソルバのハードコードパスを ARI のノード単位ワークスペースへ振り向ける; それ以外はすべて PaperBench upstream を逐語的に流れ、これにより ARI は upstream の複製タスク枠組みに整合する。この統合は section 4.5 で詳述する。

2.5 チェックポイントスコープと設計原則

ARI は 5 つの設計原則 (P1–P5) を中心に構築されている。最も拘束的なのが **P2: 決定性** である。すべての乱択ステップは乱数シードを記録し、すべての LLM 呼び出しはモデル同一性、プロンプト、出力を記録し、すべてのツール呼び出しはノード同一性でタグ付けされ (コピーオンライトのガード)、書き込みが正しいノードの作業ツリーへ落ちることを保証する。チェックポイントを再走させれば、時計依存またはモデル側非決定性として明示的にマークされた領域を除き、バイト単位で同一の成果物を再現しなければならない。

その他の原則は次の通り:

- **P1 単一成果物ツリー:** 実行のすべての出力は `$ARI_CHECKPOINT_DIR` 配下に存在する。`$HOME` や `/tmp` へ漏出しない。
- **P3 小コンポーネント:** 各スキルは独立パッケージである; スキル交換のために `ari-core` を再構築する必要はない。
- **P4 明示的系譜:** 各ノードと各チェックポイントは親を記録するため、派生実験は差分のみを再計算できる。
- **P5 コスト透明性:** コストトラッカーが各モデル呼び出しにドルコストを付与し、利用者が実行を予算で制限できるようにする。

チェックポイント直列化器は 3 つのトップレベルファイルを書き出す: 完全な探索木を `tree.json` に、ノード単位のペイロードを `nodes_tree.json` に、実行末の集計を `results.json` に。チェックポイントを跨ぐ系譜歩行は子から先祖へと辿る。

Figure 6 はチェックポイントのオンディスク形状を描く: 左に共有の実行レベルファイル、右にノード別作業ツリー。この形状が契約である: ARI の出力を入力にとる任意のツール (後続実行、リグレッションチェック、論文草稿) は、このような単一ディレクトリを指せばよい。

3 探索アルゴリズム

3.1 形式化

探索ループは木 $\mathcal{T} = (\mathcal{N}, E)$ を維持し、各ノード $n \in \mathcal{N}$ は研究の一ステップ — アイディア改訂、コード変更、ベンチマーク実行、分析など — を表す。ノードは以下を保持する:

- 離散状態 $s(n) \in \{\text{open, eval, done, failed}\}$,
- `NodeLabel` 列挙体から取られたカテゴリカルなラベル、
- 軸ごとの評価指標を保持する自由形式 dict `node.metrics`、区別されたスカラー `_scientific_score` $\in [0, 1]$ を含む、
- ノードが provisional なテキストではなく実測値を伴うかを示す Boolean `has_real_data`、ならびに
- 選択プロンプトおよび刈り込み判定で使われる帳簿フィールド `depth`。ARI は失敗ノードを再試行しない設計のため、`retry_count` シグナルは一切提示されない。

軸レベル信号からスカラー `scientific score` への集約は、`Evaluator` プロトコルの任意の実装に委譲される; ARI は固定の線形結合を `pin` しない。プロトコルは BFTS と下流のルブリック評価との唯一の継ぎ目であり、軸をスカラーへ縮約する方法について検索エンジンを `agnostic` に保つ。

Run-loop 状態. 外側ループの各反復は次のタプルを変換する:

$$S = (\mathcal{F}, \mathcal{P}, e, H),$$

ここで $\mathcal{F} \subseteq \mathcal{N}$ は拡張可能な完了ノード集合 (`done / failed`), $\mathcal{P} \subseteq \mathcal{N}$ は実行待ち (`open`) ノード集合, $e: \mathcal{N} \rightarrow \mathbb{N}$ は各フロンティアノードが何回展開されたかをカウント, $H = (l_1, \dots, l_t)$ は実行済みラベルのスライディングウィンドウ (長さ上限 $W = 20$, section 3.3 参照) である。初期状態は $S_0 = (\emptyset, \{\text{root}\}, \mathbf{0}, ())$ 。

枝刈り述語. ハードな探索予算は次の述語で表される:

$$\Pi(n; |\mathcal{N}|) = \mathbb{K}[|\mathcal{N}| \geq B_N] \vee \mathbb{K}[\text{depth}(n) \geq B_D] \vee \sigma(n), \quad (1)$$

ここで $B_N = \text{config.max_total_nodes}$, $B_D = \text{config.max_depth}$, `sterile` 述語

$$\sigma(n) = \mathbb{K}[|\Delta_n| = 0] \quad (2)$$

(Δ_n は n が追加・変更・削除したファイルの集合) は親に対して 1 ファイルも新規アーティファクトを書き出さずに終了したノードで真となる。`run-loop` がこのファイル差分を計算し、結果を `Boolean_sterile` フラグとしてノードに記録する; `should_prune` はこのフラグと 2 つの上限を読むだけである。呼び出し側が毎反復 `live` な $|\mathcal{N}|$ を渡すことで、ノード数の単一の真理源が保たれる。ステップ・コスト予算は BFTS エンジン内ではなく呼び出し側のコストトラッカーによって別途強制される。

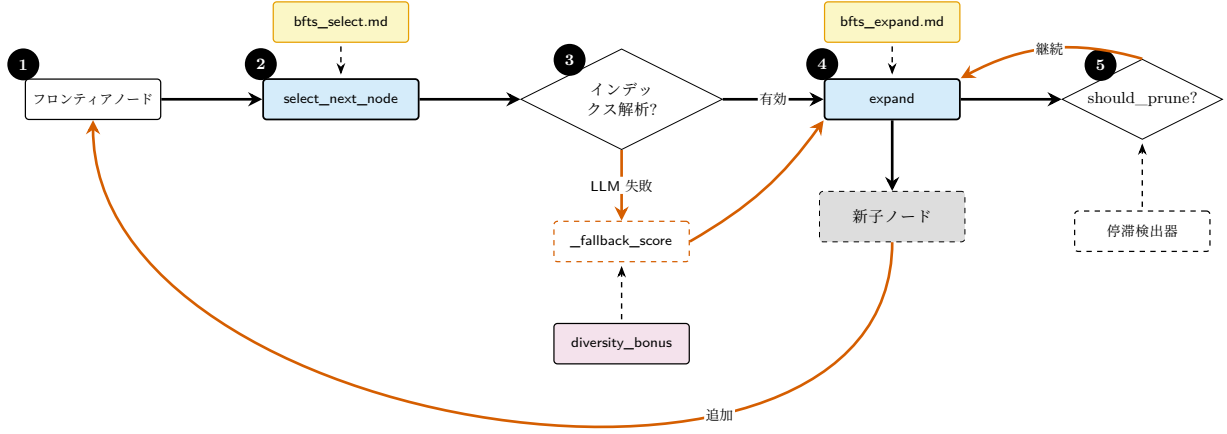


Figure 4: BFTS 制御フロー。LLM 主導の意思決定点は二つ (`select_next_node`, `expand`)、各々別個のプロンプトファイルを参照する; 選択判断は LLM が解析不能なインデックスを返したとき決定的な `_fallback_score` (`_scientific_score` と `diversity_bonus` の和) へフォールバックする。枝刈りは `should_prune`(max-total-nodes ハードカット) と停滞検出器の両方で gating される。詳細は section 3。

Retire 述語. Π に加え、run-loop はより柔軟かい **フロントティア retire 述語** ρ を適用し、最も生産的なリードでなくなった親 f を $\mathcal{F}$ から除去する:

$$\rho(f, c) = \underbrace{\mathbb{I}[r(c) > r(f)]}_{\text{規則 A}} \vee \underbrace{\mathbb{I}[e(f) \geq E_{\max}]}_{\text{規則 B}}, \quad (3)$$

ただし $E_{\max} = \text{config.max_expansions_per_node}$ (既定値 4)。規則 A は子 c が親 f を上回った瞬間に f を retire し、規則 B は親ごとの予算を制限して探索を広げる。 ρ はノードを削除しない — その履歴は $\mathcal{T}$ に残る — 再展開対象から外すだけである。

外側ループ 1 反復の段階分解 (内部展開サブグループ、並列 ReAct バッチ、実行後 retire) は fig. 7 に示す。明示的な擬似コードは section 3.2 の algorithm 1 を参照。

3.2 Run-loop アルゴリズム

algorithm 1 に明示的な擬似コードを示す。内側 while ループは空きワーカー slots を 1 展開ずつ埋め (ワーカーは次の展開提案前に必ず走り始める)、外側ブロックは `select_next_node` を 1 回 1 ノードずつ繰り返して呼んでワーカー上限までバッチを組み立て、そのバッチを並列実行する。実行後ブロックは (a) 実行されたラベルを H に追記して多様性ボーナスを実態に整合させ、(b) ρ の規則 A / B を適用する。

3.3 ノード選択 (LLM 主導)

ARI は意図的にノードのランキングを閉形式のスカラ効率に **縮約しない**。代わりに、`select_next_node` (「次のエージェントステップが実行すべき pending ノードはどれか?」) と `select_best_to_expand` (「拡張するに最も値する完了ノードはどれか?」) の双方が候補記述リストを LLM に渡し、0 始まりの単一インデックスを解析して受け取る — 各呼び出しはちょうど 1 ノードを選ぶ。各ノード n の候補記述は次の形式の一行:

```
[i] id=..., depth=..., label=...,
has_real_data=..., metrics={...},
summary=...
```

実行選択側 (`select_next_node`) は、ボーナス対象のノードに対してこの行末へ `diversity_bonus=+0.05` を付加する; `select_best_to_expand` は付加しない。これを消費するプロンプトは section B.3(選択) および section B.2(拡張対象) に逐語掲載する。プロンプトが列挙する選択基準は: `has_real_data=True` で強い `metrics` を持つノードは高価値; `metrics` は全体論的に多目的に重み付け; 優れた結果を持つ深いノードは継続に値する; `diversity_bonus` はソフトなタイブレーカとしてのみ扱う。label フィールドは `select_best_to_expand` に示される情報と一致しており、誤解を招く「low retry を優先」基準は存在しない。

多様性ボーナス

多様性ボーナス `BFTS.diversity_bonus` は最近のラベルを `_recent_label_history`(`record_run` で埋まるスライディング窓) で追跡する。当該窓内で最頻ラベルに対し相対的に**過少表現**のラベルを持つノードに対し、関数は次を返す:

$$\text{div}(n) = \begin{cases} +0.05 & \text{cnt}(n) \leq \frac{1}{2} \text{cnt}_{\max}, \\ 0 & \text{それ以外,} \end{cases} \quad (4)$$

ただし $\text{cnt}(n)$ は窓内における $\text{label}(n)$ の出現回数で、 $\text{cnt}_{\max} = \max_{\ell} \text{cnt}(\ell)$ である。0.05 という定数は意図的に小さい: `scientific score` が依然としてランキングを支配し、ボーナスはほぼ同点のものを破るのみである。

LLM フォールバック

LLM が解析不能なインデックスを返した場合、`select_next_node` は `_select_fallback` ヘルパ (候

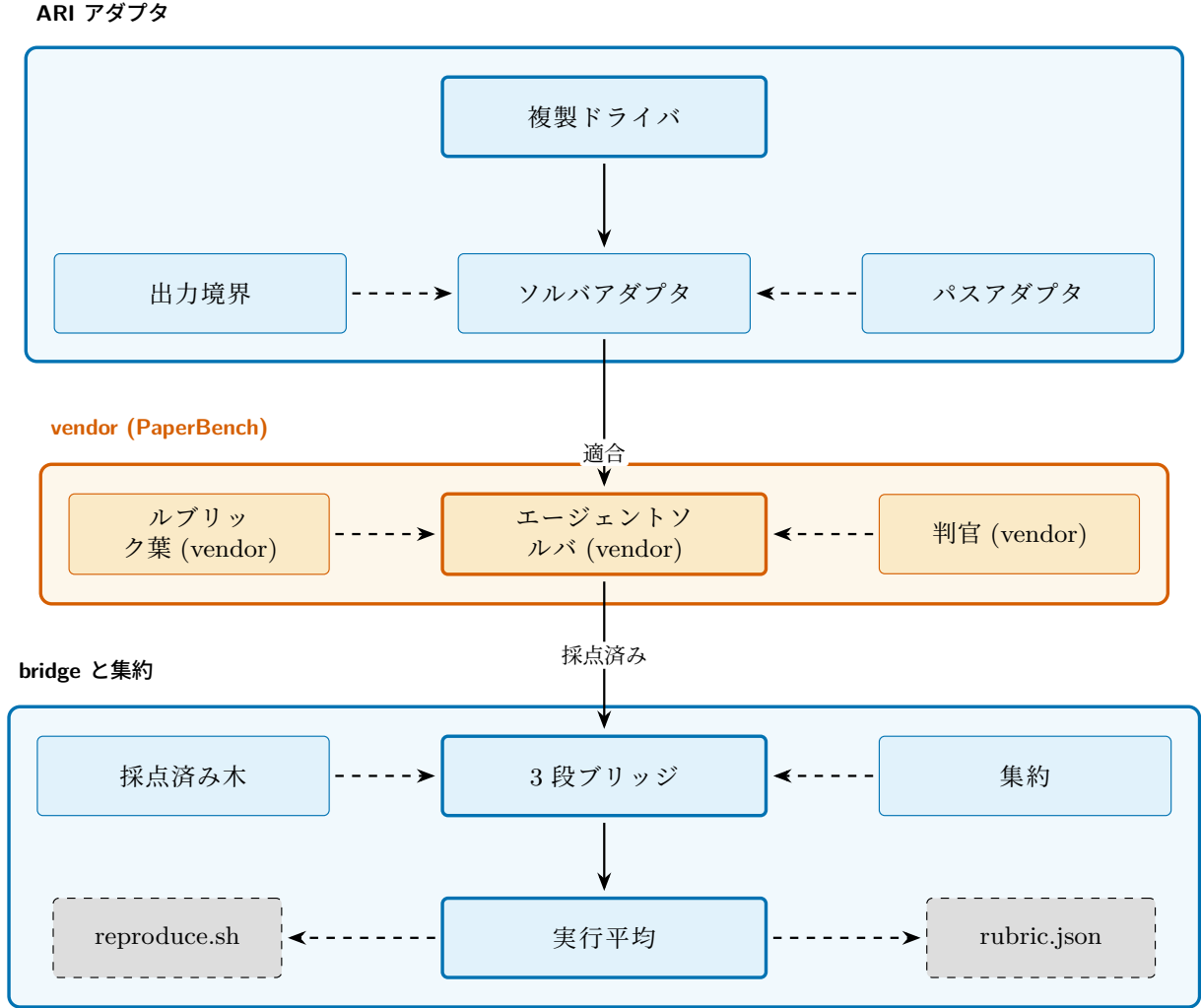


Figure 5: paper-re スキルの構成要素を 3 層で示す。最上段が ARI のアダプタ層 (複製ドライバ、ソルバアダプタ、出力境界、パスアダプタ); 中段が vendoring された PaperBench パッケージ (エージェントソルバ、ルブリック葉、判官); 下段が submission ごとの採点済み木を単一の rubric.json スコアと ARI 側の reproduce.sh に畳み込む bridge である。詳細は section 4.5。

補を `_fallback_score` 式で順位付けする) による決定的なランキングへフォールバックする。デフォルトの式は

$$fb(n) = r(n) + \text{div}(n), \quad (5)$$

ここで $r(n) \equiv \text{_scientific_score}(n)$ はノードのスカラ報酬 (section A) である。`\_select_fallback` は `has_real_data=True` のノードがあればそれを優先し、次いで $fb(\cdot)$ 最大の候補を返す。これにより、LLM 呼び出しが degrade してもループは必ず前進する。

評価層の設定

sections 3.1 and 3.3 で登場する 4 つの評価面は独立に設定可能である (デフォルト値は上式を再現するため、未変更の設定は no-op となる)。

合成式. `evaluator.composite` で指定。 $\{a_k\} \rightarrow \text{_scientific_score}$ の縮約として、加重調和平均 (デフォルト)、加重算術平均、ボトルネック型の `weighted_min`、加重幾何平均から選択。調和・幾何平均は分母を有限に保つために $\epsilon = 0.01$ で 0 軸をフロアする。

フロンティアスコア戦略. `bfts.frontier_score` で指定。eq. (5) は `scientific_plus_diversity` に相当。他の選択肢は `scientific_only` (`div` を落とす); `depth_penalized` は $\text{div}(n)$ に $-\lambda \text{depth}(n)$ を加える ($\lambda = \text{depth_penalty_lambda}$); `ucb_like` は $\text{div}(n)$ に $c_{\text{ucb}} \sqrt{\log N / (e(n) + 1)}$ を加える ($c_{\text{ucb}} = \text{ucb_c}$, $N = \sum_{n'} e(n') + |\mathcal{F}|$)。

軸セット. `evaluator.axis_mode` で指定。`dynamic` (デフォルト) は汎用 5 軸フロア + 有効 rubric + `idea.json` のプランキーワードから自動構築。`legacy` は 5 軸固定。`custom` は $\{(\text{name}, \text{description}, w)\}$ のリストをそのまま judge LLM に渡す。

選択プロンプト. `bfts.select_prompt` および `bfts.expand_select_prompt` で指定。algorithm 1 の 2 つの `selectLLM` 呼び出しで使うテンプレートを、`FilesystemPromptLoader` キーで差し替え可能。`selection` テンプレートは `experiment_goal`、`memory_context`、`candidates` の placeholder を受け取り、`expansion-target` テンプレートは `experiment_goal` と `candidates` を受け取る。

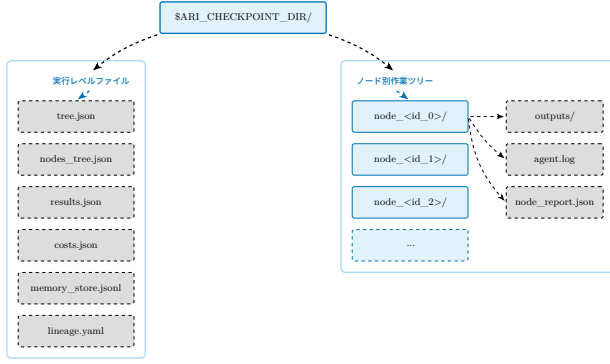


Figure 6: チェックポイントディレクトリのレイアウト。実行レベルファイル (左帯) はオーケストレータが書く; ノード別作業ツリー (右帯) は 1 BFTS 拡張内で生成された成果物を保持する。実行の再現はまさに「このディレクトリに対して再実行する」ことであり、レイアウトはそれ以外の状態が不要であることを保証する。

3.4 拡張 (1 呼び出しにつき 1 子)

expand 関数は 1 回の呼び出しで高々 1 つの新しい子を生成する。同じ親に対して複数の子が欲しい呼び出し側は expand を反復的に呼び、これまでに生成された子を existing_children 引数経由で渡すことで、LLM が重複方向を提案するのを避ける。

各新子のラベルはハードコードされたテンプレートではなく LLM の判断で決まる; 拡張プロンプト (section B.1 に逐語掲載) には以下が渡される:

- 親の状態 (succeeded / failed/no-real-data) と `_scientific_score`;
- 親の構造化された `node_report` (`delta_vs_parent` / `concerns` / `hints`, `node-report` ビルダが生成) が存在する場合、`planner` が特定の弱点を狙えるよう含める;
- 同じ深さの兄弟スコア、ならびにルートから親までの先祖スコア;
- ツリー全体の多様性指標 (これまでに見たユニークラベル、深さ分布); ならびに
- この親で既に生成された子のラベル (重複提案を避けるため)。

子は open で生成され、エージェントループに実行のため渡され (section 3.7)、すべての軸が埋まれば done に、エージェントループが例外を投げれば failed に遷移する。

3.5 枝刈りと停止

`should_prune` は枝刈り述語 Π (eq. (1)) を実装する: 総ノード上限、深さ上限、sterile フラグ。深さチェックは `max_depth` 設定を活性化する。フロンティア retire 述語 ρ (eq. (3)) は各ノードの完了後に適用される: 規則 A は子の報酬 $r(\cdot)$ が親 f を上回った時点で f を retire し、規則 B は f が $E_{\max}$ 回拡張済みになった時点で retire する。 ρ はノードを削除せず、その履歴は $\mathcal{T}$ に残るが、フロンティアのプールのみを縮めて、同じ親を掘り続けず探索を広げる。

ループは次のいずれかが満たされれば停止する:

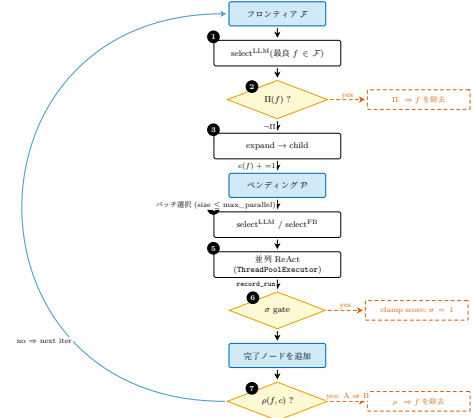


Figure 7: BFTS 外側ループ 1 反復の状態機械ビュー (上から下への単一フローチャート)。ステップ 1-3 が内部展開サブグループ、ステップ 4-7 が外側並列実行ループ。黄色のひし形は述語ガード (Π, σ, ρ)、右側の赤破線ボックスは各述語が起こす副作用 (フロンティア除去 / スコアクランプ / retire)。図中に描かない補助状態: 親ごとの展開カウンタ $e(\cdot)$ はステップ 3 で増分されステップ 7 の ρ で参照される。ラベル履歴 H (窓 W) はステップ 5 の `record_run` で追記され、ステップ 4 で多様性ボーナス `div(\cdot)` の計算に用いられる。fig. 4 (同じループを LLM プロンプト込みの細粒度制御フローとして示す) と比較されたい。

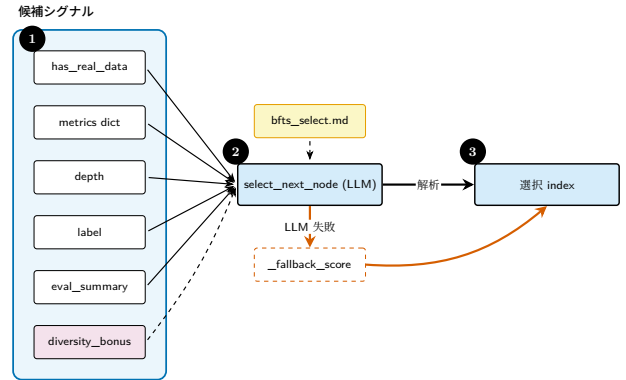


Figure 8: 次に操作するノードを選ぶ LLM に渡される選択シグナル。入力には構造化された候補記述に連結される; ソフトな `diversity_bonus` はタイブレーカであり、主シグナルではない。

- グローバル `max_total_nodes` 上限到達;
- フロンティアの全ノードが Π に該当し pending も空である;
- 呼び出し側のコストトラッカーが強制するステップ / コスト予算の枯渇;
- 停滞検出器 `detect_stagnation` が、`_scientific_score` の移動最大値がスライディング窓内で改善しないことを観測;
- 系譜判定モジュールからの `LineageDecision` が当該分岐を明示的に停止 (section B.4 参照)。

これらのうちどれが実際に最も頻出するかは経験的問いであり、凍結チェックポイントが揃い次第 section 5 で答える; 本レポートはその分布を主張しない。

Algorithm 1 BFTS run-loop(外側ループ 1 反復)。

Input: 状態 $S = (\mathcal{F}, \mathcal{P}, e, H)$; 設定 $(B_N, B_D, E_{\max})$ とワーカー上限 $\text{max_parallel} = \min(\text{max_parallel_nodes}, 4)$

Output: 更新後の状態 S'

```

1:  $\triangleright$  — 内部展開サブグループ —
2: while  $\mathcal{F} \neq \emptyset \wedge |\mathcal{P}| < \text{max\_parallel} \wedge |\mathcal{N}| < B_N$  do
3:    $f \leftarrow \text{select}_{\text{LLM}}(\mathcal{F})$   $\triangleright \text{select\_best\_to\_expand}$ 
4:   if  $\Pi(f; |\mathcal{N}|)$  then
5:      $\mathcal{F} \leftarrow \mathcal{F} \setminus \{f\}$ ; continue
6:   end if
7:    $c \leftarrow \text{expand}(f, \text{depth\_note}, \text{budget\_note})$ 
8:    $\mathcal{P} \leftarrow \mathcal{P} \cup \{c\}$ ;  $e(f) += 1$ 
9: end while
10:  $\triangleright$  — 外側並列実行バッチ —
11:  $B \leftarrow \emptyset$ 
12: while  $|B| < \min(\text{max\_parallel}, |\mathcal{P}|)$  do
13:    $n \leftarrow \text{select}_{\text{LLM}}(\mathcal{P})$   $\triangleright \text{select\_next\_node: 1 ノード}$ 
14:    $\mathcal{P} \leftarrow \mathcal{P} \setminus \{n\}$ ;  $B \leftarrow B \cup \{n\}$ 
15: end while
16: for all  $n \in B$  並列 do
17:    $n' \leftarrow \text{ReAct}(n)$   $\triangleright \text{AgentLoop.run、失敗あり}$ 
18:    $H \leftarrow (H \parallel \text{label}(n'))[-W:]$   $\triangleright \text{record\_run}$ 
19:    $\mathcal{F} \leftarrow \mathcal{F} \cup \{n'\}$ 
20:   for all  $p \in \mathcal{F} : p = \text{pa}(n')$  do  $\triangleright$  規則 A
21:     if  $r(n') > r(p)$  then
22:        $\mathcal{F} \leftarrow \mathcal{F} \setminus \{p\}$ 
23:     end if
24:   end for
25:   for all  $p \in \mathcal{F}$  do  $\triangleright$  規則 B
26:     if  $e(p) \geq E_{\max}$  then
27:        $\mathcal{F} \leftarrow \mathcal{F} \setminus \{p\}$ 
28:     end if
29:   end for
30: end for
31: return  $S' = (\mathcal{F}, \mathcal{P}, e, H)$ 

```

3.6 系譜と再現性

系譜とは最良ノードの先祖の連鎖であり、`tree.json` の `lineage` キーとしてチェックポイントへ書かれる。`LineageState` データクラスは BFTS が継続判定に用いる移動統計を保持し、跨チェックポイントの歩行器 `walk_ancestor_ckpts` は実行を跨ぐ親ポイントを提供する。アイデアプールは `get_idea_pool_for_ckpt` 経由で継承され、ランキング付きルートアイデアの選択は `RootChoice` データクラスに記録されて事後監査可能となる (プロンプト: section B.5)。

再現性は三つの不変量にかかる。第一に、すべての乱択操作はノード識別子から自身を播種する。第二に、すべてのツール呼び出しは引数と出力を該当ノードの作業ツリーへ書き、決して親へは書かない — これは MCP 層の `cow_node_id` チェックで強制される。第三に、コスト台帳が `CostTracker.init` でノードあたりのドル額を付与するため、同一プロンプトでの予算検査は決定的となる。

3.7 ReAct ドライバ

各ノードの拡張は `AgentLoop` クラスの `ReAct` 形式エージェントループを走らせる。最大ステップ数は `MAX_REACT_STEPS = 80` で、インスタンスごとに `max_react_steps` キーワードで上書きできる。別ガー

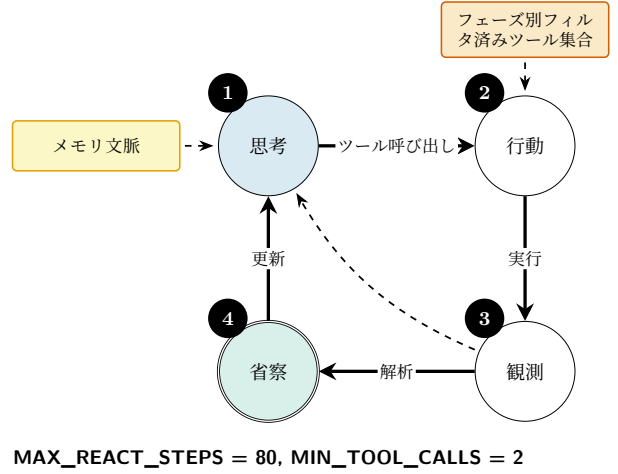


Figure 9: ReAct ループ。実線が主サイクル; 破線のバックエッジは観測がすでに開いた問いを閉じた場合の早期離脱を表す。

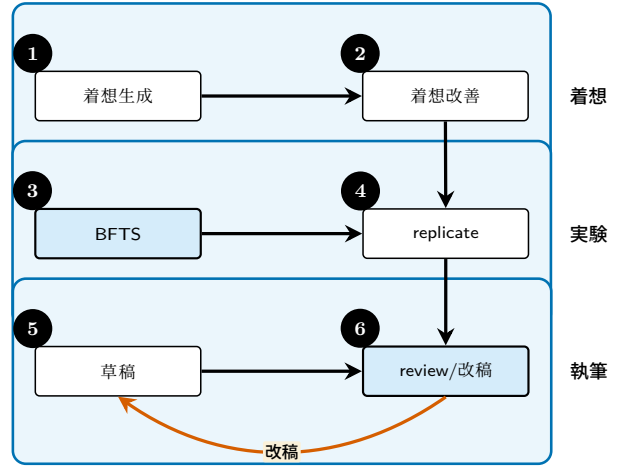


Figure 10: 論文生成スイムレーン: 着想、実験、執筆。各レーンが一つのフェーズに対応する; review から draft へのバックエッジが唯一の非単調遷移である (section 4.4)。

ド `MIN_TOOL_CALLS = 2` が trivially 短い軌跡を防ぐ。エージェントのシステムプロンプトは section B.6 に逐語掲載する。

エージェントが利用できるツール集合はツールマネージャがフェーズ別にフィルタする; 例えば探索フェーズは論文執筆ツールをマスクし、BFTS 拡張が草稿にショートカットされるのを防ぐ。一部の MCP ツールは `ari-core` 自身用に予約されており (memory CoW 操作)、LLM からは隠されるため、モデルが任意のノード ID を設定してメモリスキルのコピーオンライトチェックを迂回することはできない。

4 論文生成パイプライン

4.1 パイプライン概観

探索が停止すると、生き残った系譜が論文生成パイプライン — ARI における二本目の活動の木 (fig. 10) — に手渡される。執筆フェーズが系譜を草稿へ編纂し、レビューフェーズがその草稿をルブリックで採点する; 両者は改稿ループ (section 4.4) で結合される。パイプ

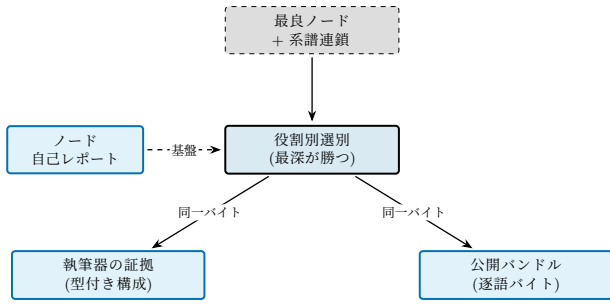


Figure 11: 証拠の組み立て。決定的な組立器は、最良ノードの系譜を一つの選定に変え、それが二つの消費者——執筆器的系譜証拠と公開成果物バンドル——へ同一の選定バイト列として流れる。ノード単位の自己レポート (内容ハッシュ、成功自己評価、逐語のビルド・実行コマンド、捕捉ハードウェア) が選定を基礎づけ、各寄与ファイルは記録メタデータにより選ばれる; パス衝突時は最深の出現が勝ち、連鎖上の削除はそのファイルを除く。

ラインは探索が残したノード単位の成果物から、執筆器が依拠してよい証拠を組み立て (section 4.2)、3つの出力を永続化する: L^AT_EX 草稿、根拠付きの葉ごとレビュー、そして本実行の採点に用いたルブリックインスタンス (ルブリックは事前固定ではなく論文ごとに生成され得るため保存する)。

4.2 証拠の組み立て

二本の木の間には決定的な証拠組立器が座る (fig. 11)。これは系譜のノード単位成果物のうち執筆器が依拠してよいものを選び、単一の科学向けレコードへ整形する。設計はこの継ぎ目を監査可能に保つ——ある主張がどのノードに辿れるかは、言語モデルの判断ではなく記録されたメタデータの関数である——ため、section 4.7 のガードレールが強制すべき具体物を持つ。

選別は役割別である: 一つのノードは三つの消費者へ異なる仕方で寄与し得て、単一の述語族が各々を判定する。ノードは、実測値を持つ (あるいは評価器がその実行を成功と認定した) とき合成集合へ、親に対しソースファイルを追加・変更したときコード集合へ、そのステップが成功したとき物語集合へ入る。行き止まりを探索しただけのノードはいずれにも入らない。各基準は記録メタデータ上の純粋述語であるため、同じ入力は常に同じ寄与集合を選ぶ。

ソースファイルは、最良ノード n^* の先祖連鎖 $\text{lin}(n^*)$ に沿って、各寄与ノードが触れたファイルを深さ順に重ね合わせて集める。子ノードは親の作業ツリーの複製内で実行されるため、一つの相対パスが複数の深さで現れ得る; 最も深い出現が勝ち、連鎖上の削除はそのファイルを除く——こうして復元集合は最良ノードの実作業ツリーに一致する。この選定は二つの消費者へ同一バイト列として供給される: 執筆器的系譜証拠と、公開成果物バンドルである。ゆえに論文がその選定から記述する系譜コードは、公開されるコードに一致する。選定は連鎖限定である——最良系譜外のノードは、その実測が合成集合で報告されても公開されず、provenance に記録される——加えて擬似コードの忠実性のため、執筆器には上位スコアノードの生ソースも別途示されるが、それ自体は公開集合ではない。

組立器はコード・パラメータ・実測値を、要約ではなく各ノードの作業ツリーから直接読む。実験が型付

Algorithm 2 証拠の組み立て (探索 → 執筆器入力)。

Input: ノード単位レポート付きの系譜ノード $\mathcal{N}$; メトリック方向写像

Output: 科学レコード $\mathcal{E}$: 型付き構成、軸ごとの極値、方法論物語

```

1:  $n^* \leftarrow \arg \max_{n \in \mathcal{N}} r(n)$  ▷ 同点: 検証済み、次に深い方
2:  $L \leftarrow \text{lin}(n^*)$  ▷ 根から最良への先祖連鎖
3:  $\mathcal{C} \leftarrow \{n \in L : \text{CONTRIBUTES}(n)\}$  ▷ 役割述語 (コード)
4:  $F \leftarrow \emptyset$  ▷ 相対パス  $\mapsto$  (ノード, 深さ)
5: for all  $n \in L$  を深さ順に do
6:   for all  $n \in \mathcal{C}$  が追加/変更したパス  $p$  do
7:     if  $p \notin F \vee \text{depth}(n) \geq \text{depth}(F[p])$  then
8:        $F[p] \leftarrow (n, \text{depth}(n))$  ▷ 最深が勝ち
9:     end if
10:  end for
11:  for all  $n$  が削除したパス  $p$  do
12:     $F$  から  $p$  を除く ▷ 削除が上書きで除去
13:  end for
14: end for
15: 各  $p \in F$  のバイトをサイズ予算の下で逐語的に読む
16: for all 実測キー  $m \notin \mathcal{I}$  do
17:    $\text{best}(m) \leftarrow \text{red}_c c[m]$  ▷ eq. (6); 決定的
18: end for
19:  $\text{NARRATE}(F)$  ▷ LLM: 逐語ソース/ログ → 散文、擬似コード
20: return  $\mathcal{E}$ 

```

き結果を宣言した場合、レコードは入力パラメータ (問題サイズ、スレッド数、シードといった構成ノブ) を、実測出力 (スループット、精度、レイテンシ) および導出量 (効率、モデル予測の上限) から分離する。この分離は装飾ではない: 構成間で報告される軸ごとの極値は、決定的に、かつ実測キーのみにに対して縮約される、

$$\text{best}(m) = \text{red}_{c \in \mathcal{E}} c[m], \quad m \notin \mathcal{I}, \quad (6)$$

ここで red はそのメトリックの宣言された方向に従う最大または最小、 $\mathcal{E}$ は寄与構成の集合、 $\mathcal{I}$ はあらゆる縮約から外される宣言済み入力キーの集合である。入力を外すことが、大きな入力規模——たとえば数百万の非零要素という問題サイズ——を主たる結果と読み違える事態を防ぐ。言語モデルは真にそれを要する唯一の仕事——逐語ソースとログを散文・擬似コードへ変換し、方法論を物語ること——に限定される。数値を供給することは決してない。

これらすべての基盤は、ノード完了時に書かれる構造化自己レポート (チェックポイント内の `node_report.json`) である。これは内容ハッシュにより、当該ノードが親に対しどのファイルを追加・変更したか; ノード自身の成功自己評価と評価器が指摘した懸念; 逐語のビルド・実行コマンド; そして実測が動いたハードウェアを記録する。ハッシュはソース選択を再現可能にし、公開バンドルが記録コマンドから構築した再現スクリプトを携えることを可能にする; 捕捉されたハードウェアは、論文の環境節を「記録なし」のままにせず事実から書けるようにする。

4.3 ルブリック形式

ルブリック R を、葉が組 $(c_i, w_i, \text{judge}_i)$ を保持する有限木としてモデル化する。各葉 i は、カテゴリ記述子 c_i 、非負の重み w_i ($\sum_i w_i = 1$ を満たす)、そして段階

的スコアを返す判官関数 $\text{judge}_i : P \rightarrow [0, 1]$ を持つ。論文スコアは凸結合

$$\text{score}(P) = \sum_i w_i \text{judge}_i(P). \quad (7)$$

で与えられる。この木構造ルブリックは PaperBench と共有されており、ARI は vendoring されたパッケージを通じて再利用する (section 4.5) ことで、採点・集約の意味論を上流に逐語的に追従させる。判官は二族が併存する: 定性的な葉のための LLM ベース判官と、二値または数値の検証器を持つ葉のための決定的判官 (例: 「コストはドル絶対値で報告されているか?」) である。両者は同一の Evaluator インタフェースを満たすため、探索エンジン (section 3) と論文パイプラインは一つの継ぎ目を介して軸をスカラーへ縮約する。

ルブリックは二つのモードのいずれかで生成される。agent-benchmark モードでは木を論文の科学的貢献で分解し、各葉は候補 submission の実行出力が論文を再現するかを問う——元来の PaperBench 枠組みである。paper-audit モードでは木の最上位の子が固定の再現性軸集合となり、各葉は論文本文そのものの記述完全性を採点する——HPC 再現性監査を意図した枠組みの反転で、問いは「エージェントは論文を再現できるか?」ではなく「論文は再現に十分な記述をしているか?」となる。venue 知識 (どの軸を、どう問うか) はエンジンに焼き込まず venue ごとのテンプレートで与えるため、ARI コアはドメイン非依存 (P4) のまま保たれ、新規 venue はコードではなくデータの変更で済む。

paper-audit スコアを退化領域——空の submission が submission 主語の葉をすべて「No」に迫りやる領域——から引き上げるには、葉の問いを (欠落した) submission ではなく論文を主語に問い直し、判官に本文と並べて論文の図を与え (視覚対応モデルが図を活用できるように)、任意の Artifact Description / Artifact Evaluation 補遺を追加入力として許容する必要があった。これらの調整はすべて ARI の wrapper 層に留まり、vendoring された判官は無改修であるため、上流バージョン間でスコアが比較可能に保たれる。論文自身の報告値から再現ログを合成することは意図的に行わない: それは executed-submission 段階を短絡させ、leaderboard 実行と比較不能な形でスコアを嵩上げしてしまう。

4.4 レビュー/改稿ループ

改稿サイクルはレビューのフィードバック $J(P_t)$ の下で P_t を P_{t+1} へ反復的に書き換える:

$$P_{t+1} = \text{Refine}(P_t, J(P_t)). \quad (8)$$

サイクルはレビューが草稿を受理した時——収束基準 $\text{score}(P_{t+1}) - \text{score}(P_t) < \epsilon$ としてモデル化——、またはセクションごとの小さな改稿上限に達した時に停止する。どちらの分岐がより頻繁に発火するかは、凍結チェックポイントが揃い次第 section 5 が答えるべき経験的問いであり、本章では定量主張を行わない。

改稿ステップは軸ごとに意図的に非単調である: 明瞭性の問題を直すと数値軸がわずかに悪化し得る。これが fig. 10 のバックエッジの源であり、収束基準を軸ごとではなく集約スコアに対して定義する理由でもある——ただし軸ごとの下限 (section 4.7) を併せて設

け、集約値が単一軸の崩壊を覆い隠さないようにしている。

4.5 PaperBench 統合

複製経路は PaperBench の再実装ではない: 上流パッケージを無改修で vendoring することで ARI はそのタスク・ルブリック意味論を逐語的に追従し、ARI はその周りの薄い wrapper のみを提供する。wrapper はプロトコル準拠のループを、共通語彙を持つ 3 つの合成可能なステージとして公開する (fig. 12):

1. *rollout* —— 複製エージェントが論文から submission (コード、および任意で再現スクリプト) へ向けて作業する;
2. *reproduce* —— submission を隔離コンテナ内で、ローカルマシン上またはクラスタヘディスパッチして実行する;
3. *grade* —— vendoring された判官が submission をルブリック木に対して採点する。

二つの設計選択がこれをベンチマークに忠実に保つ。第一に、採点は vendoring された判官に委ね、ARI のアダプタはその出力を整形し反復実行を単一スコアへ集約するだけなので、PaperBench の更新は葉採点の結合方法を変えない vendor swap となる——同じエンベロープが eq. (7) に流れ、二値葉では $\text{judge}_i \in \{0, 1\}$ となる。第二に、wrapper は *fail loud* である: ベンチマーク公正な実行が提供できない場合 (コンテナランタイム無し、スケジューラ無し、GPU リソース登録無し)、弱いホストへ黙って降格せず例外を上げる。黙った降格は結果スコアを leaderboard エントリと比較不能にするためである。旧来の silent-fallback 挙動は明示的 opt-in でのみ利用できる。

第二の関心事は、複製エージェントをホストが実際に提供するものについて誠実に保つことである。上流プロンプトに委ねると、エージェントはホストに存在しないバイナリを呼ぶ再現スクリプトを scaffold しがちで、論文の実言語に関わらず Python を既定にしがちである。ARI は両者を、scaffold 前に環境を probe するようエージェントを誘導し、利用可能なツールチェーン・GPU・ネットワーク到達性のライブ probe から生成した per-host のノートブロックを注入することで打ち消す——エージェントが読むものとホストが実際にできることを一致させる。

4.6 ドメイン幅: 実行プロファイル契約

PaperBench 上流は単一 GPU の単一コンテナを仮定する。方法論が本質的に並列な論文——多ランク MPI カーネル、多ノード学習、クラスタ固有のモジュール環境——を覆うため、ARI はルブリックを任意の実行プロファイル (fig. 13) で拡張する。プロファイルはドメイン非依存である: 論文の科学分野ではなく並列実行の形 (単一 CPU、単一/多 GPU、MPI、MPI+GPU) を、論文のスケール包絡やスケジューラヒントと共に名指す。論文が単一マシンならプロファイルは省略され、パイプラインは元の単一ノード呼び出しに退化する: 契約は加法的であって破壊的ではない。

プロファイルは二つの消費者に供給される (fig. 13)。エージェントのプロンプトには簡潔な「計算ノード実

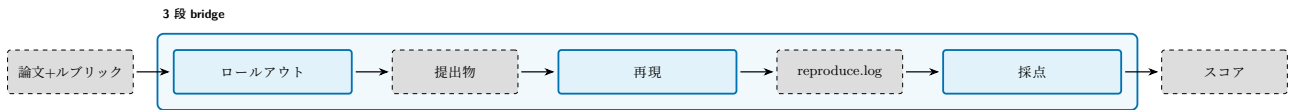


Figure 12: PaperBench 複製経路を 3 つの合成可能なステージとして示す。エージェントの *rollout* は論文とそのルブリックを submission へ変換し、*reproduce* はその submission を隔離サンドボックス内で実行して *reproduce.log* とその出力を取得し、*grade* は submission をルブリック木に対して採点する。ARI の 3 段 bridge がこれらのステージを統括し、反復実行を単一スコアへ集約する。

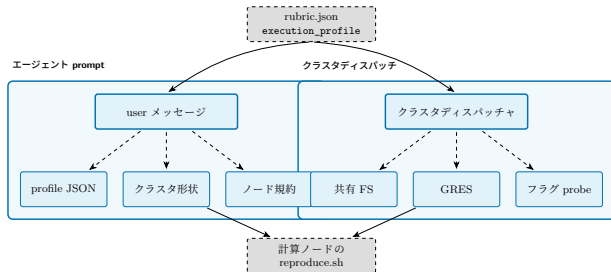


Figure 13: 実行プロファイル契約は単一のルブリック成果物から二つの消費者へ流れる: 複製エージェントのプロンプト (左) とクラスタディスパッチャ (右)。ランタイム probe がディスパッチする資源要求をローカルスケジューラに適応させるため、一つのルブリックが異種クラスタ上で無改変のまま忠実に動く。

行規約」ブロッカー—共有ファイルシステムの規律、素の MPI ランチャより スケジューラのランチャを優先、環境活性化、wall-clock ラッピング—として付加され、クラスタディスパッチ (ノード・GPU・メモリ・配置要求) を媒介する。クラスタが受理するスケジューラフラグは一致しないため、ディスパッチャは実行時にローカルスケジューラを probe し非対応フラグを落とす。こうして同じルブリックが、多 GPU クラスタでも、GPU 登録の無いサンドボックスパーティションでも、単一ワークステーションでも忠実にディスパッチする。逆向きの保証も回帰テストで強制する: 無関係なクラスタ割り当て内で動く ARI は、非 HPC 論文のプロンプトにクラスタ/MPI 固有の文言を漏らしてはならない。変わるのはこの wrapper 層だけで、vendoring されたベンチマークと ARI コアは拡張を通じて不変である。

4.7 故障様式とガードレール

4 つの故障様式は名前を与えるに足るほど頻出し、各々に助言ではなく構造的な緩和策を持つ:

- **LLM のみの裏付け:** 論文が、系譜のどのノードも裏付けない主張を述べる。緩和策は執筆者を事前選択した系譜成果物集合に制約することで、数値的・因果的主張がそれを生んだノードまで辿れるようにする; 草稿は生成されていない証拠を引用できない。
- **引用ハルシネーション:** 論文が存在しない文献を引用する。緩和策は構造的である—執筆者は引用キーを発明できず、各 bib エントリは採用前に外部インデックスに対して検証される (section 6)。
- **改稿ドリフト:** 集約スコアは上昇するが論文が一貫性を失う。緩和策は軸ごとの下限である: いず

れかの軸が閾値を下回れば改稿は中止され、ある軸での向上が別の軸の崩壊を覆い隠せない。

- **数値ドリフト:** 報告された量が、それが要約する記録済み結果と一致しない。各数値主張はそれが依拠するノードと測定値を名指す (section 4.2) ため、各々は記録済み結果から決定的に再導出され、許容誤差内で照合される; 再計算値と一致しない値はフラグされる。既定ではゲートはこの不一致を報告するのみでブロックせず、より厳格な監査ポリシーの下でのみ、最終チェックで解消されない不一致は公開ではなく却下される。

第一のガードは context も制限する: 執筆者には選択された系譜成果物と上位スコアノードのソースがサイズ予算の下で渡されるため、長時間実行でも草稿はモデルの context 窓内に収まる。

5 予備的観察

以下の観察は、稼働中のシステムがこの段階で確立していることである。これらは予備的であり、エンドツーエンドの再現事例については、制御されたベンチマークではなく単一実行によるものである; パネル規模の定量的研究は今後の課題として残す。本レポートの他所にある定量的な図は説明用であり、文書ビルドが再現可能となるよう固定シードのサンプルデータから生成している。

三つの性質はシステムの構成から直接成り立つ。コスト計上は厳密である: 各モデル呼び出しにドルコストが付与され、実行ごとにチェックポイントの結果レコードへ集計される。探索ループは、ステップ・コスト予算 ($T_{\max} / C_{\max}$) のみによってではなく、停滞検出器 (section 3.5) によって終了する。そして実行プロファイル経路は仕様どおりに振る舞う (section 4.6): ルブリック生成は、HPC 論文に対しては論文の報告するスケール包絡を伴う多ランク (MPI) 実行プロファイルを発行し、単一マシン論文に対しては省略する。一方、マルチフラグのクラスタディスパッチは、ランタイム probe が当該スケジューラの非対応フラグを落とした後、ローカルスケジューラに受理される。

エンドツーエンドでは、v0.8.0 パイプラインを非可逆圧縮の対象—GPU 上の誤差限界つき科学データ圧縮器—に対して実施した。この単一対象で達成された再現の度合いは、部分的ではあるが実質を伴うものであった。エージェントは、合成データで代替するのではなく、論文が引用するシミュレーションデータセットの実スライスを取得し; ネイティブの CUDA 補間カーネルをコンパイルして GPU 上で実行し; レベルごとの自動調律を伴う Lorenzo および多段補間予測器を実装し; 誤差限界の範囲を掃引して、およそ 50 か

ら 90 dB PSNR の再構成品質を、対応する圧縮率とともに復元した。きめ細かなルブリックに照らせば、この実行が通過したのは重み付き葉のおよそ十分の一の水準であった。その理由は、エージェントの振る舞いに根ざす二つにある: 採点された指標は CPU 予測器から得られたものであり — GPU カーネルはビルドされ実行されたものの、エージェントはその予測品質を報告結果から省いた — またエージェントは時間予算のわずかな割合を消費した後に自発的に終了し、残るデータセットと独自実装の可逆ステージを未着手のまま残した。

制御された評価 — 凍結チェックポイント上での中央値の実行コスト、シード別の探索曲線、カテゴリ別のルブリック分布、ならびに対照的な対象論文のパネルにわたるフルロールアウトの複製スコア — は、今後の課題として残す。

6 関連研究

ARI を 4 つの先行研究の流れに対して位置付ける。

6.1 コードのための木探索

AIDE [2] は ML 工学を計画・コード・デバッグ状態上の木探索として形式化する; AlphaCode [3] は競技プログラミングに対する大規模生成・濾過の規範的参照であり; MLE-bench 評価器 [4] はいまや ML 工学エージェントを計測する標準である。ARI は AIDE から BFTS の骨格を継承するが、(i) LLM 駆動の候補選択器 (section 3.3) — 閉形式のスカラー効用を固定せず、選択プロンプトが `has_real_data`、`metrics` 全体、深さ、ソフト `diversity_bonus` を一括重み付けできる —、および (ii) 実行間の系譜 (section 3.6) — AIDE も MLE-bench も形式化していない — を追加している。

6.2 自律研究エージェント

AI Scientist ファミリー [5, 6] は同様のループ (idea → experiment → paper) をエンドツーエンドで閉じるが、ルブリックと改稿サイクルを第一級オブジェクトとして露出させない。VirSci [7] は複数の科学者ペルソナを模倣し、彼らの合意で出力を採点する; Agent Laboratory [8] はエージェントを駆動者ではなく協働者として扱う。ARI は二点の運用面で異なる: **すべてがチェックポイントの中で生きる** (P1 / P4)、そしてルブリックが探索と執筆の間の契約である。

6.3 論文評価

PaperBench [9] は我々の `paper-re` 統合に最も近い類例である; それはルブリック・アズ・ツリー形式の先駆者であった。我々は同じ葉ごとの採点と集約重み (eq. (7)) を採用するが、改稿ガードとして使う軸ごとの下限で拡張している (section 4.7)。

6.4 基礎

エージェントループは ReAct パターン [1] に従う; 改稿サイクルは Self-Refine [10] と Reflexion [11] の系譜にある; 思考ステップで LLM に逐次的なスクラッチパッドを与えるという選択は Chain-of-Thought [12]

の系譜にある。これらの参照のいずれもチェックポイントスコープの状態を提案していない; それが ARI の貢献である。

初期の草稿は別の ML 研究エージェントベンチマークも引用していたが、いずれの候補ソースも我々の検証規則 (section 4.7) を通らないことが明らかになった時点で、その参照を取り除いた; 関連研究の網羅性は S2 で検証可能なエントリのみに保つ。

7 議論と限界

7.1 決定性 vs 新規性

P2、決定性原則は本システムの拘束的制約である。P2 を厳密に強制したことが、*ari-skill-memory* に「LLM 呼び出しなし」を宣言させている — それは検索の採点を、出力が遠隔サービスに依存する埋め込みモデルではなく、決定的なキーワード関数で行わなければならない。代償は、検索の質がキーワード採点で復元できる範囲に上界を持つことである。

これは偶然ではない: 再現性の勝利のすべてに、非決定性に対して扉を閉じるという代償が伴う。下流ユーザとの契約は実行ではなくチェックポイントであるため、我々はこの代償を受け入れる。

7.2 スケーリング限界

BFTS 実行ループの基盤にある単一マシン仮定は最大のスケーリング制約である。 $b > 10$ の並列拡張を望む実行には別の並行層が必要である。現状は BFTS CLI ループ内の単一の `ThreadPoolExecutor` が唯一のワーカプールである。マルチホスト拡張には系譜規律 (P4) を、先祖歩行のみではなくマージ操作も扱えるよう拡張する必要がある。

別の制約は LLM 採点者のコストである。経験的には採点者は探索器と並ぶ支配的な費用項目の一つだが、絶対ドル額で第 2 位となるか否かは凍結チェックポイントが揃ってから section 5 が答えるべき問いである。(論文ハッシュ、ルブリックハッシュ) で採点者出力をキャッシュすることは助けになる; 我々はまだこれを出荷していない。

A 記号表

本付録は本文で使われるすべての記号とその意味をまとめる。

記号	意味
$\mathcal{N}$	探索木のノード集合
$\mathcal{T}$	探索木 $(\mathcal{N}, E)$
$n \in \mathcal{N}$	個別のノード
$\text{ch}(n)$	n の子集合
$\text{pa}(n)$	n の親
$\text{depth}(n)$	n の深さ
$s(n)$	ノード状態 $\in \{\text{open}, \text{eval}, \text{done}, \text{failed}\}$
$r(n)$	n のスカラー報酬
$\mathbf{e}(n)$	軸ごとの評価ベクトル
ϕ	軸 $\rightarrow$ スカラー集約器
$\text{div}(n)$	多様性ボーナス (eq. (4))
$\text{fb}(n)$	決定的フォールバックランキング (eq. (5))
$T_{\max}, C_{\max}$	ステップ・コスト予算
R	ルブリックの木
c_i	葉 i のカテゴリ
w_i	葉 i の重み
judge_i	葉 i の判官
P	論文草稿
$\text{score}(P)$	論文集約スコア (eq. (7))
Refine	改稿演算子 (eq. (8))
ckpt	チェックポイントディレクトリ
$\text{lin}(n)$	n で終端する系譜の連鎖

B ランタイムLLMプロンプト

本付録は、v0.8.0 時点の `ari-core/ari/prompts/` から逐語複製した、ARI が実行時に使用する LLM プロンプト一覧である。各リストはディスク上のソースとバイト単位で一致している。モデルに渡るバイト列そのものが挙動を決めるため、プロンプトは翻訳せず、本レポートの各言語版で同一の原文をそのまま掲載する。

オーケストレータ

B.1 BFTS 展開

You are expanding a BFTS research tree node.

```
{goal_line}Parent node id={parent_id_short}, depth={parent_depth}, status={parent_status}
{depth_note}{budget_note}Parent metrics: {parent_metrics_json}
Parent summary: {parent_summary}
{sci_note}{idea_block}{parent_report_block}
{siblings_block}{ancestors_block}{existing_block}{diversity_block}Propose exactly ONE child research direction
that is the most scientifically valuable next step. For the "label" field, strongly prefer one of the canonical
labels [draft, improve, debug, ablation, validation]; only invent a custom label (e.g. "replication",
"generalization") when none of the five fits meaningfully — the custom string will be preserved as raw_label.
Base your choice on the experimental context above, not on a fixed template.
```

Label selection guidance:

- 'debug': parent FAILED or has no real data — diagnose and fix it.
- 'improve': parent succeeded and you want to push its metrics higher by tuning parameters, flags, or algorithms.
- 'ablation': isolate which component drives the parent's gains by removing or varying ONE component. State explicitly what is removed/varied and what delta vs. the parent metrics you expect.
- 'validation': rigorously verify the parent's claims (different seeds, edge cases, stress tests, expected-degradation checks).
- 'draft': start a fresh implementation from scratch to introduce a fundamentally NEW perspective (use this instead of inventing a new label like 'replication' or 'generalization').

Reply ONLY with a JSON array containing exactly one element: [{"label": "<one of: draft|improve|debug|ablation|validation>", "direction": "..."}]

Example: [{"label": "validation", "direction": "<one-sentence plan>"}]

B.2 BFTS 展開時選択

You are selecting which completed research node to expand next in a BFTS tree.

Experiment goal: {experiment_goal}

Completed nodes awaiting expansion:
{candidates}

Select the single most promising node to expand. Prefer nodes with high scientific_score, strong metrics, and unexplored directions. Avoid nodes that have already been retried many times or are at excessive depth. Reply with ONLY the index number (0-based).

B.3 BFTS 選択

You are selecting the most promising node to explore next in a research tree.

Experiment goal: {experiment_goal}

Relevant past memories:
{memory_context}

Candidates:
{candidates}

Selection criteria:

- Nodes with has_real_data=True and strong metrics are high-value
- Consider all metrics holistically (multi-objective)
- Deeper nodes with excellent results are worth continuing
- A small diversity_bonus is awarded to underrepresented exploration types; treat it as a soft tiebreaker, not a primary signal

Reply with ONLY the index number (0-based) of the best node.

B.4 系譜継続判定

You are a research orchestrator. Given the current state of an exploratory research run, decide the next lineage-level action. Possible actions:

- continue — current idea is still productive; keep exploring
- switch_to_idea — current idea has stagnated; switch to an alternative
- fanout — current idea succeeded; explore an alternative in parallel
- terminate — research thread is exhausted; stop

Choose the LEAST disruptive action that fits the evidence. Prefer continue unless there is a concrete reason to escalate. When choosing switch_to_idea or fanout, set target_idea_index to a value that appears in the alternatives pool. When choosing switch_to_idea, set disable_generate_ideas=true (the child should run with the chosen idea pinned, not regenerate). For fanout you may set it false so the child can also explore additional novel directions.

Reply ONLY with JSON:

{"action":str,"target_idea_index":int|null,"disable_generate_ideas":bool,"rationale":str}. No markdown.

B.5 ルートアイデア選択

You are a research orchestrator picking the ROOT idea for a run from a VirSci-generated pool. VirSci has already scored each idea by novelty/feasibility/clarity, but you have additional context (venue rubric, ancestor research thread, run notes). Your job is to pick the idea most likely to produce a strong submission for this venue, considering all signals.

Rules:

- chosen_index MUST be an integer that exists in the pool.
- Default to VirSci's choice (index 0) UNLESS another idea is clearly stronger given the additional context.
- One sentence rationale describing your reasoning.

Reply ONLY in JSON: {"chosen_index": int, "rationale": str}. No markdown.

エージェントループ

B.6 ReAct エージェントシステムプロンプト

You are a research agent. You MUST use tools to execute experiments. Do NOT write plans or text descriptions — call a tool immediately.

AVAILABLE TOOLS:

```
{tool_desc}

RULES:
- Your FIRST action must be a tool call. Never output a text plan.
- If `make_metric_spec` tool is available and this is a new experiment (not a continuation), call it early to self-determine evaluation criteria.
- NEVER fabricate numeric values — only report values from actual tool outputs
- AFTER your final measurement run, when `emit_results` is available, call it once to record a typed split between INPUT parameters (matrix size, thread count, seeds — knobs you ran on) and MEASUREMENTS (throughput, accuracy, latency — what you measured). This lets downstream stages distinguish "what we ran on" from "what we measured" so a best-of reduction never picks an input size as the result. Do NOT include input parameters in `measurements` and do NOT include measured outputs in `params`.
- When all experiments are done, return JSON: {"status": "success", "metrics": {...}, "summary": "..."}
- Do NOT call gap_analysis or generate_hypothesis
- Ensure your experiment is reproducible: capture whatever information would be needed for an independent researcher to reproduce your results and verify your findings{memory_rules}{extra}
```

評価器

B.7 ピアレビュー評価器

You are a research data extractor AND a scientific peer reviewer.
Analyze the experiment artifacts and return a JSON with:

```
has_real_data: bool (true only if numeric measurements appear in artifacts)
params: dict of INPUT/configuration values (NOT measurements). Examples: matrix size, thread count, seed.
measurements: dict of MEASURED quantities (the experiment's output). Examples: GFLOP_per_s, accuracy, latency.
metrics: dict — flat union of params and measurements (back-compat).
reason: str (one sentence describing what was measured)
{axes_block}
  comparison_found: bool (true if results involve comparison with existing approaches)
```

When scoring claim_implementation_alignment, look for concrete preconditions or contracts the plan / model states (e.g. 'eliminates RFO traffic', 'preserves equivariance', 'requires sorted input') and cross-reference them against the artifacts. Score low when the implementation contradicts a stated assumption, even if the kernel runs without errors.
Return ONLY valid JSON, no markdown fences.

B.8 メトリクス抽出

You are a research data extractor AND a scientific peer reviewer.
Analyze the experiment artifacts and return a JSON with:

```
has_real_data: bool (true only if numeric measurements appear in artifacts)
params: dict of INPUT/configuration values the experiment ran on. These are knobs, not outcomes. Examples: matrix dimensions (M, K, nnz), thread count, batch size, ISA flags, seeds. They are NOT candidates for a best-of reduction.
measurements: dict of MEASURED quantities — what a peer reviewer would treat as the experiment's result. Examples: GFLOP_per_s, GB_per_s, latency_s, accuracy. ARE candidates for best-of.
metrics: dict — for back-compat, the union of params and measurements as a single flat dict (e.g. {"GFLOP_per_s": 754.8, "M": 120000}). Downstream consumers may read either the typed split above or this flat view. NEVER include the same key in both views with different values.
reason: str (one sentence describing what was measured)
axis_scores: dict with EXACTLY these five keys, each a float in 0.0-1.0:
  - measurement_validity: are numeric measurements present and methodologically sound?
  - comparative_rigor: are results compared against baselines or prior work?
  - novelty: does the work advance beyond existing approaches?
  - reproducibility: is there enough detail for someone else to reproduce the result?
  - clarity_of_contribution: is the scientific claim stated clearly and specifically?
Score 0.0 on an axis when that dimension is absent or fatally weak. Score toward 1.0 as the dimension approaches publishable quality. These axes are combined via weighted harmonic mean, so a low score on ANY axis drags the overall score down — differentiate axes deliberately.
axis_rationales: dict with the same five keys; each value is one sentence explaining the score for that axis.
comparison_found: bool (true if results involve comparison with existing approaches)
```

Return ONLY valid JSON, no markdown fences.

パイプライン

B.9 キーワードライブラリアン

You are a research librarian. Given experiment summaries, produce a SHORT broad academic search query (3-6 words) for Semantic Scholar. Use GENERAL terms (e.g. 'algorithm optimization', not 'custom 4-stage pipelined batch-norm fused kernel'). Avoid domain-specific jargon, acronyms, or overly narrow terms. Return ONLY the query string, no explanation.

ウィザード (Viz)

B.10 ウィザード目標対話

You are an assistant helping a researcher set up an ARI experiment. ARI automatically writes, runs, benchmarks code, then produces a paper. Ask focused questions ONE AT A TIME to understand: (1) what to optimize or investigate, (2) how to measure success (metric name and direction), (3) platform/hardware constraints, (4) any baseline to compare against. Be concise. After 3-5 exchanges and sufficient info, output EXACTLY: ---READY--- followed by a complete experiment.md with sections: ## Research Goal, ## Evaluation Metric, ## Constraints. Do NOT output the MD before getting at least 2 user responses.

B.11 ウィザード設定生成器

You are helping a researcher set up an automated experiment. Convert the following research goal into a concise experiment.md file with a ## Research Goal section. Keep it to 3-5 sentences maximum. Do not add code or technical details. Research goal: {goal}

References

- [1] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: 2210.03629 [cs.CL]. URL: <https://arxiv.org/abs/2210.03629>.
- [2] Zhengyao Jiang et al. *AIDE: AI-Driven Exploration in the Space of Code*. 2025. arXiv: 2502.13138 [cs.AI]. URL: <https://arxiv.org/abs/2502.13138>.
- [3] Yujia Li et al. *Competition-Level Code Generation with AlphaCode*. 2022. DOI: 10.1126/science.abq1158. arXiv: 2203.07814 [cs.PL]. URL: <https://arxiv.org/abs/2203.07814>.
- [4] Jun Shern Chan et al. *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*. 2024. DOI: 10.48550/arXiv.2410.07095. arXiv: 2410.07095 [cs.LG]. URL: <https://arxiv.org/abs/2410.07095>.
- [5] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: <https://arxiv.org/abs/2408.06292>.
- [6] Chris Lu et al. “Towards End-to-End Automation of AI Research”. In: *Nature* 651.8107 (2026), pp. 914–919. DOI: 10.1038/s41586-026-10265-5. URL: <https://doi.org/10.1038/s41586-026-10265-5>.
- [7] Haoyang Su et al. *Many Heads Are Better Than One: Improved Scientific Idea Generation by a LLM-Based Multi-Agent System*. 2024. DOI: 10.18653/v1/2025.acl-long.1368. arXiv: 2410.09403 [cs.CL]. URL: <https://arxiv.org/abs/2410.09403>.
- [8] Samuel Schmidgall et al. *Agent Laboratory: Using LLM Agents as Research Assistants*. 2025. DOI: 10.18653/v1/2025.findings-emnlp.320. arXiv: 2501.04227 [cs.AI]. URL: <https://arxiv.org/abs/2501.04227>.
- [9] Giulio Starace et al. *PaperBench: Evaluating AI’s Ability to Replicate AI Research*. 2025. DOI: 10.48550/arXiv.2504.01848. arXiv: 2504.01848 [cs.AI]. URL: <https://arxiv.org/abs/2504.01848>.
- [10] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. DOI: 10.48550/arXiv.2303.17651. arXiv: 2303.17651 [cs.CL]. URL: <https://arxiv.org/abs/2303.17651>.
- [11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI]. URL: <https://arxiv.org/abs/2303.11366>.
- [12] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.